

CS6220 HW1

Zihe Hao

Question 1

I finished the setup by myself.

Question 2

<https://github.com/hammer-hao/CS6220HW1>

Question 3

- a. I'm using Python 3
- b. I wrote the source code within Visual Studio Code using the Python and Jupyter Lab extension.
- c. This code is ran on my machine using my global Python3 environment.
- d. I am using scikit-learn because the convenience of many built-in models and utilities.
- e. Yes. I'm running the code locally on my PC.

Question 4

```

from time import perf_counter
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Standardize then perform logic regression to predict Y
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000))
])

# Measure training time.
start = perf_counter()
pipeline.fit(X_train, y_train)
training_time = perf_counter() - start

print(f"Training complete. Training time: {training_time:.3f} seconds")

from sklearn.metrics import accuracy_score

# Measure testing/prediction time.
start = perf_counter()
y_pred = pipeline.predict(X_test)
testing_time = perf_counter() - start
accuracy = accuracy_score(y_test, y_pred)

print(f"Prediction time: {testing_time:.6f} seconds")
print(f"Test accuracy: {accuracy:.2%}")

```

Training complete. Training time: 0.006 seconds

Prediction time: 0.001273 seconds
Test accuracy: 93.33%

Question 5

```

# Load features and the class column.
iris = fetch_ucirepo(id=53)
X = iris.data.features
y = iris.data.targets["class"]

[1] ✓ 2.1s

from sklearn.model_selection import train_test_split

# 80% training, 20% testing split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
X_train.head(5)

[6] ✓ 0.0s

...

```

	sepal length	sepal width	petal length	petal width
8	4.4	2.9	1.4	0.2
106	4.9	2.5	4.5	1.7
76	6.8	2.8	4.8	1.4
9	4.9	3.1	1.5	0.1
89	5.5	2.5	4.0	1.3

```

from time import perf_counter
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Standardize then perform logic regression to predict Y
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000))
])

# Measure training time.
start = perf_counter()
pipeline.fit(X_train, y_train)
training_time = perf_counter() - start

[4] ✓ 0.1s

```

Question 6

In [8]: `from ucimlrepo import fetch_ucirepo`

```

# Load features and the class column.
iris = fetch_ucirepo(id=53)
X = iris.data.features
y = iris.data.targets["class"]

```

In [9]: `from sklearn.model_selection import train_test_split`

```

# 80% training, 20% testing split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
X_train.head(5)

```

Out[9]:

	sepal length	sepal width	petal length	petal width
8	4.4	2.9	1.4	0.2
106	4.9	2.5	4.5	1.7
76	6.8	2.8	4.8	1.4
9	4.9	3.1	1.5	0.1
89	5.5	2.5	4.0	1.3

```
In [10]: from time import perf_counter
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Standardize then perform logic regression to predict Y
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000))
])

# Measure training time.
start = perf_counter()
pipeline.fit(X_train, y_train)
training_time = perf_counter() - start\

print(f"Training complete. Training time: {training_time:.3f} seconds")
```

Training complete. Training time: 0.005 seconds

```
In [11]: from sklearn.metrics import accuracy_score

# Measure testing/prediction time.
start = perf_counter()
y_pred = pipeline.predict(X_test)
testing_time = perf_counter() - start
accuracy = accuracy_score(y_test, y_pred)

print(f"Prediction time: {testing_time:.6f} seconds")
print(f"Test accuracy: {accuracy:.2%}")
```

Prediction time: 0.001025 seconds

Test accuracy: 93.33%

Question 6: report

- The training time ranges from 0.005 to 0.006 seconds.
- The testing/prediction time is just over 0.001 second
- The test accuracy on the test data is 93.33%